



IT-SIMPLERA INSTITUTE

Cyber Security Analyst Internship Program

WEEK 03 ASSIGNMENT REPORT

Cybersecurity Fundamentals & Security Concepts

Prepared By:	Muhammad Tayyab Riaz
Role:	Red Team Intern
Registration No:	RT-2605
Institute	IT-Simplera Institute
Supervisor	Talal Rehan
Week	Week 03
Submission Date	Saturday, 18 July 2026

Table of Contents

Table of Contents	2
Introduction	3
Task 1 – Protocols and Port Numbers.....	3
Task 2 – HTTP vs HTTPS Analysis.....	4
Task 3 – Kali Linux Network Reconnaissance.....	5
Connectivity Test – ping.....	5
Service Discovery – nmap	6
Task 4 – What Happens When You Type a URL and Press Enter	9
Task 5 – Networking Cheat Sheet	10
Task 6 – Kali Linux Web Enumeration.....	10
WHOIS Findings.....	10
Technology Enumeration – whatweb	12
Interesting Findings.....	19
Challenges	19
Conclusion.....	20
References	20

Introduction

This report covers Week 03 of the Cyber Security Analyst Internship at IT-Simplera Institute. The focus this week was networking fundamentals – the protocols that move traffic around the internet, the difference between a plain HTTP connection and an encrypted HTTPS one, and a first hands-on pass at reconnaissance using Kali Linux.

The tasks below combine research (protocols, ports, the URL-to-webpage process) with practical exercises (ping, nmap, whois, whatweb) run against authorised, publicly permitted targets. Screenshots from my own Kali VM are included where the task calls for live command output.

Task 1 – Protocols and Port Numbers

Fifteen protocols that come up constantly in day-to-day network and security work, with their default port, transport layer, purpose, and whether they're secure out of the box.

Protocol	Port	TCP/UDP	Purpose	Secure / Insecure
FTP	21 (control) / 20 (data)	TCP	Transfers files between a client and a server	Insecure – credentials and data sent in plain text
SSH	22	TCP	Secure remote login, command execution and file transfer (SFTP/SCP)	Secure – traffic is encrypted end-to-end
Telnet	23	TCP	Remote login to another device over a network	Insecure – everything, including passwords, is sent in plain text
SMTP	25	TCP	Sends email between mail servers	Insecure by default – needs STARTTLS / port 587 for encryption
DNS	53	TCP/UDP	Resolves domain names into IP addresses	Insecure by default – queries are unencrypted unless DoH/DoT is used
DHCP	67 / 68	UDP	Automatically assigns IP addresses and network settings to devices	Insecure – no built-in authentication, vulnerable to rogue servers
HTTP	80	TCP	Loads web pages	Insecure – data is transmitted in plain text
POP3	110	TCP	Downloads email from a mail server to a client	Insecure – unless POP3S (995) is used

Protocol	Port	TCP/UDP	Purpose	Secure / Insecure
NTP	123	UDP	Synchronises device clocks across a network	Insecure – can be abused for DDoS amplification
IMAP	143	TCP	Retrieves and manages email directly on the mail server	Insecure – unless IMAPS (993) is used
SNMP	161 / 162	UDP	Monitors and manages network devices such as routers and switches	Insecure in v1/v2c – community strings sent in plain text
LDAP	389	TCP	Queries and maintains directory services such as Active Directory	Insecure – unless LDAPS (636) is used
HTTPS	443	TCP	Loads web pages over an encrypted connection	Secure – encrypted with TLS/SSL
SMB	445	TCP	Shares files, folders and printers on a network	Insecure by default – a common exploit target (e.g. EternalBlue)
RDP	3389	TCP	Provides graphical remote desktop access to Windows machines	Insecure by default – frequently brute-forced when exposed to the internet

Task 2 – HTTP vs HTTPS Analysis

I checked ten widely-used websites for three things: which protocol they load on, whether typing the plain http:// version redirects to https://, and what that actually says about the connection's security.

Website	Protocol	Auto-Redirect?	Notes
google.com	HTTPS	Yes	Enforces HSTS; certificate issued via Google Trust Services
youtube.com	HTTPS	Yes	Runs on the same secure infrastructure as Google
amazon.com	HTTPS	Yes	Encrypts checkout and payment data end-to-end
wikipedia.org	HTTPS	Yes	Wikimedia moved to HTTPS-only for every request back in 2015
github.com	HTTPS	Yes	Enforces HSTS and rejects plain HTTP for the main site
facebook.com	HTTPS	Yes	Meta enforces HTTPS across all of its products
linkedin.com	HTTPS	Yes	Protects login credentials and messages in transit
microsoft.com	HTTPS	Yes	Uses a Microsoft-issued certificate validated by the browser
yahoo.com	HTTPS	Yes	Redirects immediately even when HTTP is typed in manually

Website	Protocol	Auto-Redirect?	Notes
x.com	HTTPS	Yes	Enforces HSTS to block downgrade attacks

What “secure” actually means here

All ten sites redirected straight to HTTPS, which is what you'd expect from any mainstream site in 2026. The security comes from the TLS handshake that runs before any page data is exchanged: the browser and server agree on a cipher suite, the server proves its identity with a certificate signed by a trusted Certificate Authority, and both sides derive a symmetric session key that encrypts everything from that point on. That's why a padlock in the address bar means the connection can't be read or silently modified in transit – it doesn't say anything about whether the site itself is trustworthy, just that the channel to it is private.

Task 3 – Kali Linux Network Reconnaissance

Connectivity Test – ping

Command used:

```
(kali@kali)-[~]
$ ping google.com
PING google.com (172.217.160.142) 56(84) bytes of data.
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=1 t
tl=110 time=352 ms
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=2 t
tl=110 time=336 ms
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=3 t
tl=110 time=338 ms
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=4 t
tl=110 time=337 ms
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=5 t
tl=110 time=337 ms
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=6 t
tl=110 time=336 ms

64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=7 t
tl=110 time=336 ms
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=8 t
tl=110 time=341 ms
64 bytes from maa03s29-in-f14.1e100.net (172.217.160.142): icmp_seq=9 t
tl=110 time=335 ms
^C
— google.com ping statistics —
10 packets transmitted, 9 received, 10% packet loss, time 9005ms
rtt min/avg/max/mdev = 335.330/338.951/352.217/4.935 ms
```

ping google.com

Field	Result
Destination	google.com → 172.217.160.142 (maa03s29-in-f14.1e100.net)
Packets sent / received	10 sent / 9 received (stopped manually with Ctrl+C)
Packet loss	10%
Min / Avg / Max RTT (ms)	335.330 / 338.951 / 352.217
TTL value	110

Service Discovery – nmap

scanme.nmap.org is the Nmap project's own public test host, set up specifically so people can practice scanning legally. Scanning any other host without permission is not something to do outside a lab – I only pointed nmap at this authorised target and at my own machine's local IP.

Commands used:

nmap scanme.nmap.org

```
(kali@kali)-[~]
$ nmap scanme.nmap.org

Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-18 09:39 EDT
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.29s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff
:fe18:bb2f
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
100/tcp   filtered newacct
427/tcp   filtered svrloc
8087/tcp  open  simplifymedia
9929/tcp  open  nping-echo
31337/tcp open  Elite

Nmap done: 1 IP address (1 host up) scanned in 10.90 seconds
```

nmap -sV scanme.nmap.org

```
(kali㉿kali)-[~]
$ nmap -sV scanme.nmap.org

Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-18 09:39 EDT
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.29s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff
:fe18:bb2f
Not shown: 993 closed tcp ports (reset)
PORT      STATE      SERVICE      VERSION
22/tcp    open      ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open      http         Apache httpd 2.4.7 ((Ubuntu))
100/tcp   filtered  newacct
427/tcp   filtered  svrloc
8087/tcp  open      http         BusyBox httpd 1.13
9929/tcp  open      nping-echo   Nping echo
31337/tcp open      tcpwrapped
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 26.63 seconds
```

nmap -sV 192.168.0.107

```
(kali㉿kali)-[~]
$ nmap -sV 192.168.0.107

Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-18 09:55 EDT
Nmap scan report for 192.168.0.107
Host is up (0.0000030s latency).
All 1000 scanned ports on 192.168.0.107 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.33 seconds
```

scanme.nmap.org – default scan

Field	Result
Open ports	22 (ssh), 80 (http), 8087 (simplifymedia), 9929 (nping-echo), 31337 (Elite)
Filtered ports	100 (newacct), 427 (svrloc) – blocked by a firewall/ACL, state unconfirmed
Scan time	10.90 seconds

scanme.nmap.org – with -sV (version detection)

Field	Result
22/tcp	OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (protocol 2.0)

Field	Result
80/tcp	Apache httpd 2.4.7 (Ubuntu)
8087/tcp	BusyBox httpd 1.13 (relabelled from "simplifymedia" once actually probed)
9929/tcp	Nping echo
31337/tcp	tcpwrapped – service present but not positively identified
Scan time	26.63 seconds – more than double the default scan, since -sV sends extra probes

Local IP address

Field	Result
Local IP	192.168.0.107 (eth0, via ip a)
Open ports found	[Run: nmap -sV 192.168.0.107 and insert the results here]
Services running	Not shown: 1000 closed tcp ports (reset)

Which open port is the biggest risk, and why

- Port 22 (OpenSSH 6.6.1p1) & Port 80 (Apache 2.4.7): These versions shipped with Ubuntu 14.04 in 2014. Because they are over a decade old, they contain multiple publicly documented CVEs (vulnerabilities).
- The Value of -sV: A default scan only shows that the ports are "open," whereas -sV identifies the exact software versions needed to check against a vulnerability database.
- Port 31337: This is not a real backdoor; it is a "leet" (elite) hacker culture Easter egg intentionally left open on scanme.nmap.org. However, it serves as a reminder to always investigate non-standard ports.

Bonus – what -sV does

- **How -sV Works:** Once a port is found open, Nmap probes the service and matches its response against a fingerprint database to identify the exact software and version.
- **The Benefit:** It prevents false assumptions. In this scan, it corrected a default port guess of "simplifymedia" on port 8087 to the actual service: BusyBox httpd 1.13.
- **The Trade-off:** Version detection is slower; here, it increased the scan time by roughly 2.5x (26.63s vs 10.90s).

Task 4 – What Happens When You Type a URL and Press Enter

A quick walkthrough of everything that happens between hitting Enter and the page showing up on screen.

1. DNS Lookup

The browser first checks its own cache, then the OS cache, then asks a DNS resolver (usually your ISP's, or a public one like 1.1.1.1 or 8.8.8.8). If none of those have an answer cached, the resolver works its way from the root servers to the .com (or other) TLD servers to the domain's authoritative name server, which finally returns the IP address.

2. IP Address Resolution

The resolver hands the browser the actual IPv4 or IPv6 address tied to that domain, and the browser caches it for as long as the record's TTL allows so it doesn't have to repeat the lookup on the next request.

3. TCP Three-Way Handshake

Before any data moves, the browser and server establish a reliable connection: the client sends a SYN packet, the server replies with SYN-ACK, and the client answers with ACK. This happens on port 80 for HTTP or port 443 for HTTPS.

4. TLS/HTTPS Handshake

For an HTTPS site, the client and server then negotiate a TLS version and cipher suite, the server presents its certificate, and the browser checks it against a trusted Certificate Authority. Both sides use this to agree on a symmetric session key, and every byte exchanged after this point is encrypted.

5. HTTP Request

With the secure channel in place, the browser sends an HTTP request – typically a GET – including headers like cookies, the user-agent string, and accepted content types, asking the server for the specific page or resource.

6. Server Response

The server processes the request (which might mean querying a database or running application logic), then sends back a status code, response headers, and the actual content – usually HTML, plus references to CSS, JavaScript, and images.

7. Browser Rendering

The browser parses the HTML into a DOM tree and the CSS into a CSSOM, combines them into a render tree, executes any JavaScript, fetches additional resources it finds along the way, and finally paints the finished page to the screen.

Task 5 – Networking Cheat Sheet

A condensed reference sheet pulling together the protocols from Task 1 with their real-world risk and the safer alternative to reach for.

Protocol	Port	TCP/UDP	Purpose	Security Risk	Secure Alternative
FTP	21/20	TCP	File transfer	Plain-text credentials & data	SFTP / FTPS
SSH	22	TCP	Encrypted remote access	Brute-force if exposed openly	Key-based auth + fail2ban
Telnet	23	TCP	Remote login	No encryption at all	SSH
SMTP	25	TCP	Send email	Plain text, spoofing	SMTPS (587/465) with TLS
DNS	53	TCP/UDP	Name resolution	Cache poisoning, no integrity check	DNSSEC / DNS-over-HTTPS
DHCP	67/68	UDP	IP auto-assignment	Rogue DHCP servers	DHCP snooping / 802.1X
HTTP	80	TCP	Load web pages	Data interception (MITM)	HTTPS
POP3	110	TCP	Download email	Plain-text credentials	POP3S (995)
NTP	123	UDP	Clock sync	DDoS amplification, spoofing	NTS (Network Time Security)
IMAP	143	TCP	Manage email on server	Plain-text credentials	IMAPS (993)
SNMP	161/162	UDP	Device monitoring	Default community strings	SNMPv3
LDAP	389	TCP	Directory services	Plain-text bind credentials	LDAPS (636)
HTTPS	443	TCP	Secure web pages	Weak ciphers if misconfigured	Keep certs & TLS config current
SMB	445	TCP	File/printer sharing	Unauthenticated access, exploits	SMB3 with encryption enabled
RDP	3389	TCP	Remote desktop	Brute-force, internet exposure	VPN + Network Level Auth + MFA

Task 6 – Kali Linux Web Enumeration

Passive information gathering on two public domains using whois (domain registration records) and whatweb (web technology fingerprinting).

WHOIS Findings

google.com

```

(kali@kali)-[~]
$ whois google.com
Domain Name: GOOGLE.COM
Registry Domain ID: 2138514_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-07-18T13:31:04Z <<<

```

Field	Value
Registrar	MarkMonitor Inc.
Creation Date	15 September 1997
Registry Expiry Date	14 September 2028
Name Servers	ns1.google.com, ns2.google.com, ns3.google.com, ns4.google.com
DNSSEC	Unsigned
Registrant Organisation	Google LLC (contact details redacted for privacy)

github.com

```
(kali@kali)-[~]
$ whois github.com
Domain Name: GITHUB.COM
Registry Domain ID: 1264983250_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2024-09-07T09:16:32Z
Creation Date: 2007-10-09T18:20:50Z
Registry Expiry Date: 2026-10-09T18:20:50Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Name Server: DNS1.P08.NSONE.NET
Name Server: DNS2.P08.NSONE.NET
Name Server: DNS3.P08.NSONE.NET
Name Server: DNS4.P08.NSONE.NET
Name Server: NS-1283.AWSDNS-32.ORG
Name Server: NS-1707.AWSDNS-21.CO.UK
Name Server: NS-421.AWSDNS-52.COM
Name Server: NS-520.AWSDNS-01.NET
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-07-18T13:31:49Z <<<
```

Field	Value
Registrar	MarkMonitor Inc.
Creation Date	9 October 2007
Registry Expiry Date	9 October 2026
Name Servers	dns1–dns4.p08.nsone.net, ns-1283.awsdns-32.org, ns-1707.awsdns-21.co.uk, ns-421.awsdns-52.com, ns-520.awsdns-01.net
DNSSEC	Unsigned
Registrant Organisation	GitHub, Inc.

Technology Enumeration – whatweb

Commands used:

whatweb -v google.com

```

(kali@kali)-[~]
$ whatweb -v google.com
WhatWeb report for http://google.com
Status : 301 Moved Permanently
Title : 301 Moved
IP : 142.250.67.238
Country : UNITED STATES, US

Summary : HTTPServer[gws], RedirectLocation[http://www.google.com/],
UncommonHeaders[content-security-policy-report-only], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]

Detected Plugins:
[ HTTPServer ]
    HTTP server header string. This plugin also attempts to
    identify the operating system from the server header.

    String : gws (from server string)

[ RedirectLocation ]
    HTTP Server string location. used with http-status 301 and
    302

    String : http://www.google.com/ (from location)

[ UncommonHeaders ]
    Uncommon HTTP server headers. The blacklist includes all
    the standard headers and many non standard but common ones.
    Interesting but fairly common headers should have their own
    plugins, eg. x-powered-by, server and x-aspnet-version.
    Info about headers can be found at www.http-stats.com

    String : content-security-policy-report-only (from header
s)

[ X-Frame-Options ]
    This plugin retrieves the X-Frame-Options value from the
    HTTP header. - More Info:
    http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.
    aspx

    String : SAMEORIGIN

[ X-XSS-Protection ]
    This plugin retrieves the X-XSS-Protection value from the
    HTTP header. - More Info:
    http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.
    aspx

    String : 0

```



```

HTTP Headers:
  HTTP/1.1 301 Moved Permanently
  Location: http://www.google.com/
  Content-Type: text/html; charset=UTF-8
  Content-Security-Policy-Report-Only: object-src 'none';base-uri
'self';script-src 'nonce-Bp9-5FRtV2TN9Z80BPjJg' 'strict-dynamic' 'rep
ort-sample' 'unsafe-eval' 'unsafe-inline' https: http;;report-uri https
://csp.withgoogle.com/csp/gws/other-hp
  Date: Sat, 18 Jul 2026 13:32:25 GMT
  Expires: Mon, 17 Aug 2026 13:32:25 GMT
  Cache-Control: public, max-age=2592000
  Server: gws
  Content-Length: 219
  X-XSS-Protection: 0
  X-Frame-Options: SAMEORIGIN
  Connection: close

WhatWeb report for http://www.google.com/
Status      : 200 OK
Title       : Google
IP          : 142.251.157.119
Country     : UNITED STATES, US

Summary     : Cookies[AEC,NID,__Secure-STRP], HTML5, HTTPServer[gws], Htt
pOnly[AEC,NID], Script, UncommonHeaders[content-security-policy-report-
only], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]

Detected Plugins:
[ Cookies ]
  Display the names of cookies in the HTTP headers. The
  values are not returned to save on space.

  String      : __Secure-STRP
  String      : AEC
  String      : NID

[ HTML5 ]
  HTML version 5, detected by the doctype declaration

[ HTTPServer ]
  HTTP server header string. This plugin also attempts to
  identify the operating system from the server header.

  String      : gws (from server string)

[ HttpOnly ]
  If the HttpOnly flag is included in the HTTP set-cookie
  response header and the browser supports it then the cookie
  cannot be accessed through client side script - More Info:
  http://en.wikipedia.org/wiki/HTTP_cookie

  String      : AEC,NID

```

```

[ Script ]
  This plugin detects instances of script HTML elements and
  returns the script language/type.

[ UncommonHeaders ]
  Uncommon HTTP server headers. The blacklist includes all
  the standard headers and many non standard but common ones.
  Interesting but fairly common headers should have their own
  plugins, eg. x-powered-by, server and x-aspnet-version.
  Info about headers can be found at www.http-stats.com

String      : content-security-policy-report-only (from header
s)

[ X-Frame-Options ]
  This plugin retrieves the X-Frame-Options value from the
  HTTP header. - More Info:
  http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.aspx

String      : SAMEORIGIN

[ X-XSS-Protection ]
  This plugin retrieves the X-XSS-Protection value from the
  HTTP header. - More Info:
  http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.aspx

String      : 0

HTTP Headers:
  HTTP/1.1 200 OK
  Content-Type: text/html; charset=ISO-8859-1
  Date: Sat, 18 Jul 2026 13:32:28 GMT
  Expires: -1
  Cache-Control: private, max-age=0
  Content-Security-Policy-Report-Only: object-src 'none';base-uri
'self';script-src 'nonce-VAXtuM4ieueY4rJfIuqGdA' 'strict-dynamic' 'rep
ort-sample' 'unsafe-eval' 'unsafe-inline' https: http;;report-uri https
://csp.withgoogle.com/csp/gws/other-hp
  P3P: CP="This is not a P3P policy! See g.co/p3phelp for more in
fo."
  Content-Encoding: gzip
  Server: gws
  Content-Length: 27479
  X-XSS-Protection: 0
  X-Frame-Options: SAMEORIGIN
  Set-Cookie: __Secure-STRP=ANmZwa20i9T_M-mEPrmUyIz-quP-5r6xM4FfG
70e2_RJIzTanTDkt2f4_OhKSw7MOXA1ChFXsKk8JtA6iWz69AptEjaRN4F9x0ik; expire
s=Sat, 18-Jul-2026 13:37:28 GMT; path=/; domain=.google.com; Secure; Sa
meSite=strict
  Set-Cookie: AEC=AdJVEavkWCEXP08aRGh_WIb25arSS5b_MvE-zH5U7100qSu
F04ZWZWphTg; expires=Thu, 14-Jan-2027 13:32:28 GMT; path=/; domain=.goo
gle.com; Secure; HttpOnly; SameSite=lax
  Set-Cookie: NID=533=Uwds0U-T33w-rhHs1-B1WIaY-ud9gjhBFI0sWh7CDhY
JQrbpwXhPOHjZF5gmBSwb8COVA8yA3YBxdzXyWwxIk5xlDomi1-8j1bKqz0WqTwsrLnvrvH

```

whatweb -v github.com

```

(kali@kali)-[~]
└─$ whatweb -v github.com
WhatWeb report for http://github.com
Status      : 301 Moved Permanently
Title       : <None>
IP          : 20.207.73.82
Country     : UNITED STATES, US

Summary     : RedirectLocation[https://github.com/]

Detected Plugins:
[ RedirectLocation ]
  HTTP Server string location. used with http-status 301 and 302
  String      : https://github.com/ (from location)

HTTP Headers:
  HTTP/1.1 301 Moved Permanently
  Content-Length: 0
  Location: https://github.com/
  connection: close

WhatWeb report for https://github.com/
Status      : 200 OK
Title       : GitHub · Change is constant. GitHub keeps you ahead. · GitHub
IP          : 20.207.73.82
Country     : UNITED STATES, US

Summary     : Content-Language[en-US], Cookies[_gh_sess,_octo,logged_in],
  Email[you@domain.com], HTML5, HTTPServer[github.com], HttpOnly[_gh_sess,logged_in],
  Open-Graph-Protocol[object][1401488693436528], OpenSearch[/opensearch.xml],
  Script[application/json,module], Strict-Transport-Security[max-age=31536000;
  includeSubdomains;preload], UncommonHeaders[x-content-type-options,referrer-policy,content-security-policy,x-github-request-id],
  X-Frame-Options[deny], X-XSS-Protection[0]

Detected Plugins:
[ Content-Language ]
  Detect the content-language setting from the HTTP header.
  String      : en-US

[ Cookies ]
  Display the names of cookies in the HTTP headers. The values are not returned to save on space.
  String      : _gh_sess
  String      : _octo
  String      : logged_in

[ Email ]
  Extract email addresses. Find valid email address and syntactically invalid email addresses from mailto: link tags. We match syntactically invalid links containing mailto: to catch anti-spam email addresses, eg. bob at gmail.com. This uses the simplified email regular expression from http://www.regular-expressions.info/email.html for valid email address matching.
  String      : you@domain.com

[ HTML5 ]
  HTML version 5, detected by the doctype declaration

```



```

[ HTTPServer ]
  HTTP server header string. This plugin also attempts to
  identify the operating system from the server header.

  String      : github.com (from server string)

[ HttpOnly ]
  If the HttpOnly flag is included in the HTTP set-cookie
  response header and the browser supports it then the cookie
  cannot be accessed through client side script - More Info:
  http://en.wikipedia.org/wiki/HTTP_cookie

  String      : _gh_sess,logged_in

[ Open-Graph-Protocol ]
  The Open Graph protocol enables you to integrate your Web
  pages into the social graph. It is currently designed for
  Web pages representing profiles of real-world things .
  things like movies, sports teams, celebrities, and
  restaurants. Including Open Graph tags on your Web page,
  makes your page equivalent to a Facebook Page.

  Version     : object
  Module      : 1401488693436528

[ OpenSearch ]
  This plugin identifies open search and extracts the URL.
  OpenSearch is a collection of simple formats for the
  sharing of search results.

  String      : /opensearch.xml

[ Script ]
  This plugin detects instances of script HTML elements and
  returns the script language/type.

  String      : application/json,module

[ Strict-Transport-Security ]
  Strict-Transport-Security is an HTTP header that restricts
  a web browser from accessing a website without the security
  of the HTTPS protocol.

  String      : max-age=31536000; includeSubdomains; preload

[ UncommonHeaders ]
  Uncommon HTTP server headers. The blacklist includes all
  the standard headers and many non standard but common ones.
  Interesting but fairly common headers should have their own
  plugins, eg. x-powered-by, server and x-aspnet-version.
  Info about headers can be found at www.http-stats.com

  String      : x-content-type-options,referrer-policy,content-s
  ecurity-policy,x-github-request-id (from headers)

[ X-Frame-Options ]
  This plugin retrieves the X-Frame-Options value from the
  HTTP header. - More Info:
  http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.
  aspx

  String      : deny

[ X-XSS-Protection ]
  This plugin retrieves the X-XSS-Protection value from the
  HTTP header. - More Info:
  http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.
  aspx

```

```

HTTP Headers:
  HTTP/1.1 200 OK
  Date: Sat, 18 Jul 2026 13:32:54 GMT
  Content-Type: text/html; charset=utf-8
  Vary: X-PJAX, X-PJAX-Container, Turbo-Visit, Turbo-Frame, X-Requested-With, Accept-Language, Sec-Fetch-Site, Accept-Encoding, Accept, X-Requested-With
  content-language: en-US
  ETag: W/"777063d02b5ce7eeffddd48488ce65a6"
  Cache-Control: max-age=0, private, must-revalidate
  Strict-Transport-Security: max-age=31536000; includeSubdomains; preload
  X-Frame-Options: deny
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Referrer-Policy: origin-when-cross-origin, strict-origin-when-cross-origin
  Content-Security-Policy: default-src 'none'; base-uri 'self'; child-src github.githubassets.com github.com/assets-cdn/worker/ github.com/assets/ gist.github.com/assets-cdn/worker/; connect-src 'self' uploads.github.com www.githubstatus.com collector.github.com raw.githubusercontent.com api.github.com github-cloud.s3.amazonaws.com github-production-repository-file-5c1aeb.s3.amazonaws.com github-production-upload-manifest-file-7fdce7.s3.amazonaws.com github-production-user-asset-6210df.s3.amazonaws.com *.rel.tunnels.api.visualstudio.com wss://*.rel.tunnels.api.visualstudio.com github.githubassets.com objects-origin.githubusercontent.com copilot-proxy.githubusercontent.com proxy.individual.githubcopilot.com proxy.business.githubcopilot.com proxy.enterprise.githubcopilot.com *.actions.githubusercontent.com wss://*.actions.githubusercontent.com productionresultssa0.blob.core.windows.net productionresultssa1.blob.core.windows.net productionresultssa2.blob.core.windows.net productionresultssa3.blob.core.windows.net productionresultssa4.blob.core.windows.net productionresultssa5.blob.core.windows.net productionresultssa6.blob.core.windows.net productionresultssa7.blob.core.windows.net productionresultssa8.blob.core.windows.net productionresultssa9.blob.core.windows.net productionresultssa10.blob.core.windows.net productionresultssa11.blob.core.windows.net productionresultssa12.blob.core.windows.net productionresultssa13.blob.core.windows.net productionresultssa14.blob.core.windows.net productionresultssa15.blob.core.windows.net productionresultssa16.blob.core.windows.net productionresultssa17.blob.core.windows.net productionresultssa18.blob.core.windows.net productionresultssa19.blob.core.windows.net github-production-repository-image-32fea6.s3.amazonaws.com github-production-release-asset-2e65be.s3.amazonaws.com insights.github.com wss://alive.github.com wss://alive-staging.github.com m api.githubcopilot.com api.individual.githubcopilot.com api.business.githubcopilot.com api.enterprise.githubcopilot.com wss://production-copilot-host.webpubsub.azure.com edge.fullstory.com rs.fullstory.com; font-src github.githubassets.com; form-action 'self' github.com gist.github.com copilot-workspace.githubnext.com objects-origin.githubusercontent.com; frame-ancestors 'none'; frame-src viewscreen.githubusercontent.com notebooks.githubusercontent.com www.youtube-nocookie.com; img-src 'self' data: blob: github.githubassets.com media.githubusercontent.com camo.githubusercontent.com identicons.github.com avatars.githubusercontent.com private-avatars.githubusercontent.com github-cloud.s3.amazonaws.com objects.githubusercontent.com release-assets.githubusercontent.com secured-user-images.githubusercontent.com user-images.githubusercontent.com private-user-images.githubusercontent.com opengraph.githubassets.com marketplace-screenshots.githubusercontent.com copilotprodattachments.blob.core.windows.net/github-production-copilot-attachments/ github-production-user-asset-6210df.s3.amazonaws.com customer-stories-feed.github.com spotlights-feed.github.com explore-feed.github.com objects-origin.githubusercontent.com *.githubusercontent.com images.ctfassets.net/8aevphvgewt8/; manifest-src 'self'; media-src github.com user-images.githubusercontent.com secured-user-images.githubusercontent.com private-user-images.githubusercontent.com github-production-user-asset-6210df.s3.amazonaws.com gist.github.com github.githubassets.com assets.ctfassets.net/8aevphvgewt8/ videos.ctfassets.net/8aevphvgewt8/; script-src github.githubassets.com; style-src 'unsafe-inline' github.githubassets.com; upgrade-insecure-requests; worker-src github.githubassets.com github.com/asset-cdn/worker/ github.com/assets/ gist.github.com/assets-cdn/worker/

```

google.com

Field	Finding
Redirect behaviour	http://google.com returns a 301 to http://www.google.com/ – still plain HTTP. The jump to HTTPS happens client-side via HSTS preloading, not at the server
Server header	gws (Google Web Server) – custom, no version disclosed

Field	Finding
Content-Security-Policy	Sent as report-only – violations are logged, not blocked
X-Frame-Options	SAMEORIGIN – framing allowed from the same origin only
X-XSS-Protection	0 – deliberately disabled (deprecated, superseded by CSP)
Cookies	AEC, NID, __Secure-STRP – HttpOnly and Secure flags set correctly
CMS / framework detected	None – consistent with custom, in-house infrastructure

github.com

Field	Finding
Redirect behaviour	http://github.com redirects with a single 301 straight to https://github.com/ – HTTPS is enforced at the server, not left to the browser
Strict-Transport-Security	max-age=31536000; includeSubdomains; preload – sent on every response, reinforcing the preload list rather than relying on it alone
Content-Security-Policy	Fully enforced (default-src 'none' plus an explicit allowlist) – blocks unlisted sources instead of just logging them
X-Frame-Options	deny – all framing blocked, stricter than Google's SAMEORIGIN
Server header	github.com – custom, no version disclosed
Cookies	_gh_sess, _octo, logged_in – all Secure, HttpOnly, SameSite=Lax
Hosting clue	Response IP falls in Microsoft Azure's range – consistent with GitHub running on Azure since Microsoft's 2018 acquisition
False positive	WhatWeb's Email plugin flagged “you@domain.com” – a placeholder string in the page markup, not a real harvested address

Interesting Findings

Both domains use MarkMonitor as their registrar rather than a standard consumer registrar – that's a common pattern for large tech companies, since MarkMonitor specialises in brand and domain protection and makes unauthorised transfers much harder to pull off. Neither domain has DNSSEC enabled, which is a small but genuine gap even for organisations this size. The 10-year gap between google.com's 1997 registration and github.com's 2007 registration also lines up neatly with each company's founding timeline.

Challenges

The trickiest part this week was keeping the TLS handshake steps in the right order in my head – it's easy to blur the TCP handshake and the TLS handshake into one step when they're actually two separate exchanges stacked on top of each other. Reading whois output for a heavily protected domain also took

a second look, since a lot of the registrant contact fields are redacted for privacy and it's not obvious at first that this is normal rather than missing data. Scoping the nmap scans to only authorised targets also took some care – it's a good habit to build early rather than something to get casual about.

Conclusion

This week connected the theory of ports and protocols to what's actually happening on the wire when a browser loads a page, and gave a first real pass at using Kali's reconnaissance tools against authorised targets. The main takeaway is that most everyday protocols (FTP, Telnet, HTTP, POP3, IMAP) were never designed with encryption in mind, and the secure alternatives (SFTP, SSH, HTTPS, IMAPS) exist specifically to patch that gap – recognising which is which is a big part of what a security analyst actually does day to day.

References

- IANA – Service Name and Transport Protocol Port Number Registry – iana.org/assignments/service-names-port-numbers
- Nmap Official Reference Guide – nmap.org/book/man.html
- WhatWeb (urbanadventurer) – github.com/urbanadventurer/WhatWeb
- ICANN Lookup (WHOIS) – lookup.icann.org
- Cloudflare Learning Center – cloudflare.com/learning
- Mozilla Developer Network – TLS/SSL overview – developer.mozilla.org
- OWASP Foundation – owasp.org
- Kali Linux Tools Documentation – kali.org/tools
- Scanme.Nmap.Org – authorised public scanning target – scanme.nmap.org